Homework 07

Eric Chen

All code is in Python (NumPy, pandas, scikit-learn, matplotlib). Seeds: Question 3 uses NumPy seed 432; Question 4 uses seed 432 for the test split and CV folds.

Question 1: LDA

Estimate π_k, $\boldsymbol{\mu}_k$ and the pooled $\boldsymbol{\Sigma}$ (denominator $n - K$) for `SAheart` (`chd` response, nine predictors, `famhist` coded 1/2); classify the training data with the hand-computed scores; compare with `LinearDiscriminantAnalysis(solver="svd")`.

(a) Estimates. $\hat{\pi}_k = n_k/n$, $\hat{\boldsymbol{\mu}}_k = n_k^{-1} \sum_{y_i=k} \mathbf{x}_i$, and $\hat{\boldsymbol{\Sigma}} = (n-K)^{-1} \sum_k \sum_{y_i=k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^T$.

```
heart = pd.read_csv("data/saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})
X = heart.drop(columns="chd").to_numpy(); y = heart["chd"].to_numpy()
names = heart.drop(columns="chd").columns
n, p = X.shape; classes = np.unique(y); K = len(classes)
pi = np.array([np.mean(y == k) for k in classes])                    # K
mu = np.vstack([X[y == k].mean(axis=0) for k in classes])           # K x p
Sigma = sum((X[y == k] - mu[i]).T @ (X[y == k] - mu[i])
            for i, k in enumerate(classes)) / (n - K)               # p x p
print("n =", n, " p =", p, " priors (chd = 0, 1):", pi.round(4))
print("shapes: X", X.shape, " y", y.shape, " mu", mu.shape, " Sigma", Sigma.shape)
```

```
n = 462  p = 9  priors (chd = 0, 1): [0.6537 0.3463]
shapes: X (462, 9)  y (462,)  mu (2, 9)  Sigma (9, 9)
```

Class means:

	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age
chd = 0	135.46	2.63	4.34	23.97	1.32	52.37	25.74	15.93	38.85
chd = 1	143.74	5.52	5.49	28.12	1.6	54.49	26.62	19.15	50.29

Pooled covariance matrix (2 decimals):

	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age
sbp	405.43	14.59	4.58	49.16	0.34	−15.59	18.94	64.4	95.13
tobacco	14.59	19.24	0.76	7.54	0.02	−2.06	1.83	20.52	22.77
ldl	4.58	0.76	4	6.03	0.09	0.34	2.66	−2.53	6.48
adiposity	49.16	7.54	6.03	56.75	0.43	−5.31	22.71	16.12	60.51
famhist	0.34	0.02	0.09	0.43	0.23	0.08	0.18	0.77	1
typea	−15.59	−2.06	0.34	−5.31	0.08	95.57	2.64	7.96	−20.28
obesity	18.94	1.83	2.66	22.71	0.18	2.64	17.62	4.69	15.7
alcohol	64.4	20.52	−2.53	16.12	0.77	7.96	4.69	598.28	27.89
age	95.13	22.77	6.48	60.51	1	−20.28	15.7	27.89	184.13

(b) In-sample classification. The score for class k at every row of $\mathbf{X}$ is the n-vector $\mathbf{X}\hat{\boldsymbol{\Sigma}}^{-1}\hat{\boldsymbol{\mu}}_k - \frac{1}{2}\hat{\boldsymbol{\mu}}_k^T\hat{\boldsymbol{\Sigma}}^{-1}\hat{\boldsymbol{\mu}}_k + \log\hat{\pi}_k$; stacking both classes gives an $n \times K$ score matrix.

```
Sinv = np.linalg.inv(Sigma)                                          # p x p
scores = X @ Sinv @ mu.T - 0.5 * np.sum((mu @ Sinv) * mu, axis=1) + np.log(pi)
```

```
assert scores.shape == (n, K)
pred_hand = classes[scores.argmax(axis=1)]
cm_hand = pd.crosstab(pd.Series(y, name="Observed"), pd.Series(pred_hand, name="Predicted"))
print(cm_hand, f"\ntraining error = {np.mean(pred_hand != y):.4f}")
```

```
Predicted    0   1
Observed
0          258  44
1           73  87
training error = 0.2532
```

(c) Library LDA.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
lda = LDA(solver="svd").fit(X, y)                  # empirical priors by default
pred_lib = lda.predict(X)
print(pd.crosstab(pd.Series(y, name="Observed"), pd.Series(pred_lib, name="Predicted")))
print("identical labels:", np.array_equal(pred_lib, pred_hand),
      "| same priors:", np.allclose(lda.priors_, pi), "| same means:", np.allclose(lda.means_, mu))
```

```
Predicted    0   1
Observed
0          258  44
1           73  87
identical labels: True | same priors: True | same means: True
```

Yes, the results match: both give 258 + 87 = 345 correct and 117 misclassified cases (training error 0.253), and the two label vectors are identical for all 462 observations. The SVD solver never forms $\hat{\Sigma}^{-1}$ explicitly, but it uses the same priors, means, and $n - K$ pooled covariance, so it gives the same discriminant rule.

Question 2: QDA and marginal screening

Fit QDA (`QuadraticDiscriminantAnalysis(tol=1e-8)`) on the 40, then the 10, highest-variance pixels of `zip.train`, and evaluate on `zip.test`.

```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis as QDA
train = pd.read_csv("data/zip-train.csv.gz", header=None).to_numpy()
test = pd.read_csv("data/zip-test.csv.gz", header=None).to_numpy()
ytr, Xtr = train[:, 0].astype(int), train[:, 1:]
yte, Xte = test[:, 0].astype(int), test[:, 1:]
order = np.argsort(-Xtr.var(axis=0, ddof=1))       # training-data variances only

def screen_qda(m):
    keep = order[:m]
    qda = QDA(tol=1e-8).fit(Xtr[:, keep], ytr)
    pred = qda.predict(Xte[:, keep])
    cm = pd.crosstab(pd.Series(yte, name="Observed"), pd.Series(pred, name="Predicted"))
    return cm, np.mean(pred != yte), np.mean(qda.predict(Xtr[:, keep]) != ytr), keep

cm40, err40, tr40, keep40 = screen_qda(40)
print(cm40, f"\nm = 40: test error = {err40:.4f}, training error = {tr40:.4f}")
print("class sizes:", np.bincount(ytr), "\nsmallest eigenvalue of each class covariance:",
      [f"{np.linalg.eigvalsh(np.cov(Xtr[ytr == k][:, keep40].T)).min():.1e}" for k in range(10)])
```

```
Predicted    0    1    2    3    4    5    6    7    8    9
Observed
0          319    0   24    6    2    2    1    0    5    0
1            0  238    5    0   11    0    7    0    3    0
2            8    0  168    8    8    3    0    0    3    0
3            5    0    4  128    0   14    0    0   15    0
4            0    2   13    1  150    1    0    1    7   25
5           12    0    3    5    1  131    0    0    6    2
6            0    0    8    0    1    8  148    0    5    0
7            1    0    6    1    6    0    0  124    2    7
8            1    0   10   12    3    2    0    0  137    1
9            0    0    1    2    7    1    1    4    9  152
m = 40: test error = 0.1555, training error = 0.0833
class sizes: [1194 1005  731  658  652  556  664  645  542  644]
smallest eigenvalue of each class covariance: ['1.9e-03', '2.7e-07', '1.5e-02', '1.1e-02', '1.0e-02',
'1.7e-02', '3.4e-03', '2.3e-03', '3.3e-02', '5.7e-03']
```

(a) The 40-pixel QDA has test misclassification rate **0.155** (312 of 2,007 test digits); the confusion matrix is above.

(b) Yes, the method works. With all 256 pixels, each class covariance has 32,896 free entries, and many pixels are constant within a class, so the matrices are singular. With 40 pixels each $\hat{\Sigma}_k$ has $40 \cdot 41/2 = 820$ entries. Every class has at least 542 training images, which is more than 40, and the screen keeps the central pixels where the ink varies. So all ten matrices are invertible. The digit-1 matrix is close to singular (smallest eigenvalue $\approx 3 \times 10^{-7}$) because a "1" barely touches some of these pixels, but `tol=1e-8` accepts it and 1s are still classified well. The test error of about 15.5% is much better than chance (90%). For comparison, LDA on the same 40 pixels has test error about 23% (not shown). Class-specific covariances help here because each digit's strokes produce its own pixel correlations. QDA still overfits: its training error is 8.3%, about half the test error, because it estimates about 8,600 parameters. The main confusions are $4 \longrightarrow 9$, $0 \longrightarrow 2$, and $3 \longrightarrow 5/8$, which are digits that differ mainly in pixels this variance screen leaves out.

```
cm10, err10, tr10, _ = screen_qda(10)
print(cm10, f"\nm = 10: test error = {err10:.4f}, training error = {tr10:.4f}")
```

```
Predicted    0    1   2    3   4   5   6    7   8    9
Observed
0          298    0  11    1   1  16  28    4   0    0
1            0  241  10    0   3   1   1    0   2    6
2           19    2  95   14  14   3  22   21   5    3
3            5    0   6  103   3  42   0    0   7    0
4            1    5  14    0  95   2   4   15  17   47
5           15    0   6   25   4  96   4    2   8    0
6           41    0  15    4   0  18  78    7   5    2
7            4    1   6    0   6   0   1  122   0    7
8            1    2  11   36  12  11   4    1  74   14
9            0    2   9    2  20   0   0   16  10  118
m = 10: test error = 0.3423, training error = 0.2886
```

(c) With 10 pixels the test error rises to **0.342**, more than twice the 40-pixel error. The training error (0.289) is also high and close to the test error, so the loss comes from bias, not variance. Ten pixels cannot separate ten digit shapes: 0 and 6, 3 and 5, and 4 and 9 look alike in these few central pixels. Cutting the parameters from 820 to 55 per class lowers estimation variance, but the 40-pixel model was not variance-limited enough for this to matter. The screen is also unsupervised. High marginal variance shows where ink varies overall, not which pixels separate the classes. Each removed pixel loses real discriminating information, and this outweighs the small reduction in variance.

Question 3: Compare LDA and QDA

Two equally likely classes; (X_1, X_2) normal with means $(\mp 1, 0)$ and correlations $\pm\rho$; $X_3, \ldots, X_{10}$ independent $N(0,1)$ noise; 100 training and 1,000 test observations per class; equal priors.

```
rng = np.random.default_rng(432)                    # seed 432
rhos = [0, 0.2, 0.4, 0.6, 0.8, 0.9]

def simulate(n, rho):
    X = rng.standard_normal((2 * n, 10)); y = np.repeat([1, 2], n)
    for k, r, m1 in [(1, rho, -1.0), (2, -rho, 1.0)]:
        L = np.linalg.cholesky([[1, r], [r, 1]]); idx = y == k
        X[idx, :2] = X[idx, :2] @ L.T + [m1, 0]       # correlate X1, X2; shift mean
    return X, y

def one_run(rho):                                   # same data for both methods
    Xa, ya = simulate(100, rho); Xb, yb = simulate(1000, rho)
    return [np.mean(M(priors=[0.5, 0.5]).fit(Xa, ya).predict(Xb) != yb) for M in (LDA, QDA)]

single = pd.DataFrame([one_run(r) for r in rhos], index=pd.Index(rhos, name="rho"),
                      columns=["LDA", "QDA"])         # (a) one dataset per rho
reps = np.array([[one_run(r) for _ in range(50)] for r in rhos])   # (b) 6 x 50 x 2
avg = pd.DataFrame(reps.mean(axis=1), index=single.index, columns=["LDA", "QDA"])
se = pd.DataFrame(reps.std(axis=1, ddof=1) / np.sqrt(50), index=single.index, columns=["LDA", "QDA"])
print(pd.concat({"(a) one dataset": single, "(b) mean of 50": avg, "(b) SE": se}, axis=1).round(4))
```

	(a) one dataset		(b) mean of 50		(b) SE	
	LDA	QDA	LDA	QDA	LDA	QDA
rho						
0.0	0.1505	0.1695	0.1723	0.1980	0.0014	0.0018
0.2	0.1670	0.1920	0.1710	0.1977	0.0014	0.0018
0.4	0.1710	0.1850	0.1691	0.1906	0.0015	0.0016
0.6	0.1675	0.1845	0.1688	0.1768	0.0013	0.0017
0.8	0.1730	0.1355	0.1680	0.1364	0.0016	0.0010
0.9	0.1620	0.0925	0.1691	0.1007	0.0013	0.0010

```
okabe = {"LDA": ("#0072B2", "o", "-"), "QDA": ("#D55E00", "s", "--")}
fig, ax = plt.subplots(figsize=(6.5, 4))
for m, (c, mk, ls) in okabe.items():
    ax.errorbar(rhos, avg[m], yerr=se[m], color=c, marker=mk, ms=5, ls=ls, lw=1.8, capsize=3)
    ax.annotate(m, (rhos[-1], avg[m].iloc[-1]), xytext=(8, 0), textcoords="offset points",
                color=c, fontsize=10, va="center", fontweight="bold")
ax.axhline(0.1587, color="0.6", lw=1, ls=":")
ax.annotate("Bayes error at ρ = 0, Φ(−1) = 0.159", xy=(0.0, 0.1587), xytext=(0.12, 0.125),
            fontsize=9, color="0.35", arrowprops=dict(arrowstyle="->", color="0.5", lw=0.8, shrinkB=4))
ax.set_xlabel("True within-class correlation, ρ", fontsize=10)
ax.set_ylabel("Average test misclassification rate", fontsize=10)
ax.set_xticks(rhos); ax.set_xlim(-0.05, 0.98); ax.set_ylim(0.08, 0.215)
ax.tick_params(labelsize=9); ax.grid(alpha=0.25, lw=0.6)
ax.spines[["top", "right"]].set_visible(False)
ax.set_title("50 replications per ρ; 100 training and 1,000 test observations per class; seed 432",
             loc="left", fontsize=9, color="0.35", pad=4)
ax.annotate("LDA wins at weak correlation; QDA wins at strong correlation", xy=(0, 1),
```

```
            xycoords="axes fraction", xytext=(0, 17), textcoords="offset points",
            ha="left", va="bottom", fontsize=12)
fig.tight_layout(); plt.show()
```

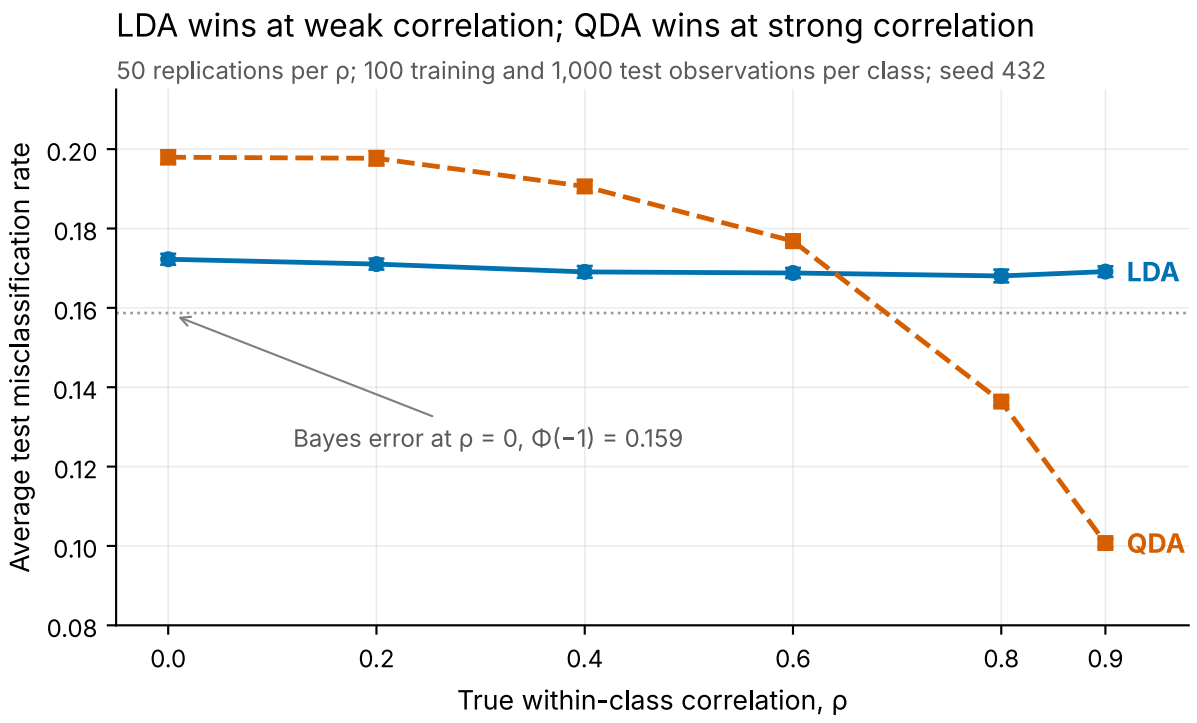


Figure 1: Average test misclassification rate of LDA and QDA (±1 SE over 50 replications).

(c) At $\rho = 0$ the two class covariances are both $\mathbf{I}$, so LDA's shared-covariance assumption is exactly true and the Bayes boundary ($x_1 = 0$) is linear. LDA (0.172) is about 2.6 points better than QDA (0.198), and both are above the Bayes error 0.159. LDA's error stays near 0.17 for every ρ. As ρ grows, the true covariances $\begin{pmatrix} 1 & \pm\rho \\ \pm\rho & 1 \end{pmatrix}$ differ more, and the Bayes boundary becomes strongly quadratic. LDA cannot use the opposite correlation signs: its pooled covariance averages them to about $\mathbf{I}$, so it keeps a linear boundary and gains nothing. QDA's error falls to 0.136 at $\rho = 0.8$ and 0.101 at $\rho = 0.9$. The curves cross between $\rho = 0.6$ and 0.8, where QDA's error is 0.177 and LDA's is 0.169 at $\rho = 0.6$. This is the bias-variance trade-off. LDA estimates $10 \cdot 11/2 = 55$ covariance parameters and QDA estimates 110, from only 100 observations per class. For small ρ, LDA's bias is small and QDA's extra variance costs more. For large ρ, LDA's bias, from fitting a linear boundary to a curved problem, is larger than QDA's variance.

The eight noise covariates matter because the method cannot know they are noise. QDA estimates their means, their variances, and all their correlations separately in each class. With 100 observations per class, the two estimated 8×8 blocks (and their cross-covariances with X_1, X_2) differ only by chance. QDA reads these chance differences as real and builds quadratic terms along irrelevant directions, which adds variance to the boundary and test error. LDA also estimates small spurious mean differences and correlations in the noise directions, which tilt its linear boundary, but it pools both classes and estimates half as many covariance parameters, so it is hurt less. That is why QDA's advantage appears only when ρ is large.

Question 4: LDA/QDA analysis skill

Analysis report: predicting coronary heart disease with LDA/QDA

Data and goal. `SAheart` has $n = 462$ men, 9 numeric predictors (`famhist` coded 1/2), and response `chd` (34.6% cases). The goal is the lowest misclassification rate within LDA/QDA methods. Seed 432 was used for a stratified 75/25 split into 346 training and 116 test observations. The test set was not used until the final evaluation.

Candidates and CV. The candidates were plain LDA and QDA; PCA (standardize, then keep the first q components, $q = 1, \ldots, 9$) followed by LDA or QDA; and a blend $w\,\hat{P}_{\text{LDA}} + (1 - w)\hat{P}_{\text{QDA}}$ with $w \in \{0, 0.1, \ldots, 1\}$. Every candidate used the same stratified 10 training folds. Scaling and PCA were refit inside each fold. The rule was to select the lowest mean CV error and report its SE (SD of the fold errors$/\sqrt{10}$).

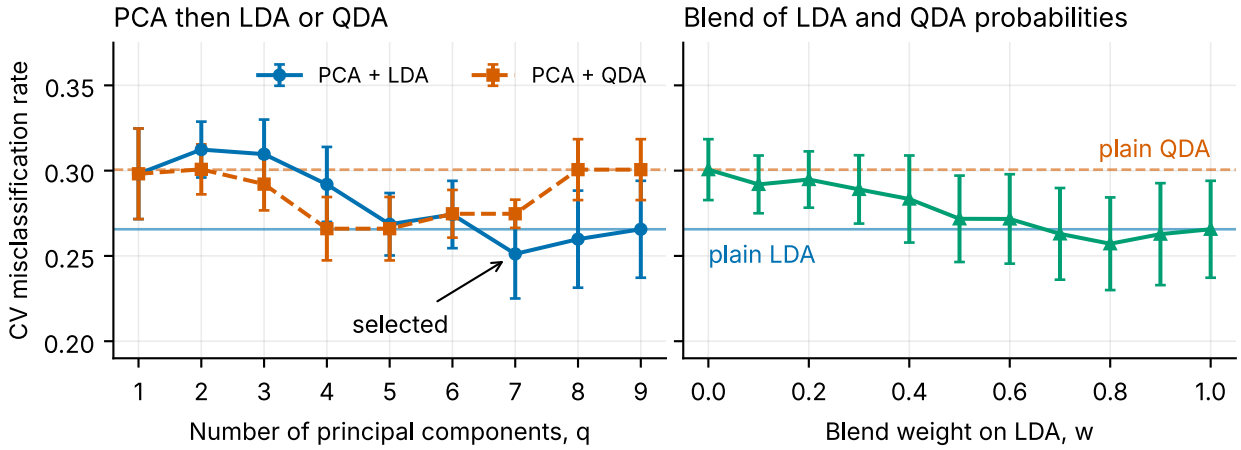


Figure: Mean 10-fold CV error ±1 SE (346 training cases). Horizontal lines are plain LDA (solid) and QDA (dashed).

Model	Setting	CV error	SE	Within 1 SE
PCA + LDA	**q = 7 PCs**	**0.251**	**0.026**	**selected**
Blend	w = 0.8	0.257	0.027	yes
LDA	all 9 predictors	0.266	0.028	yes
PCA + QDA	q = 4 PCs	0.266	0.019	yes
QDA	all 9 predictors	0.301	0.018	no

Selected model and test result. The selected model is **PCA + LDA with q = 7 PCs** (CV error 0.251 ± 0.026). After refitting on all 346 training cases, its single test evaluation gives a misclassification rate of **0.267** (95% Wilson CI 0.195–0.354). Test confusion matrix (observed rows 0/1, predicted columns 0/1): [65, 11; 20, 20]. References, not used for selection: majority class 0.345, plain LDA 0.276, plain QDA 0.276.

Findings and limitations.

- A shared covariance works better here: LDA-type models are on top, and plain QDA (CV 0.301) is the only candidate more than one SE worse than the selected one. QDA's extra 2×45 covariance terms add variance without enough curvature in the signal to pay for them. Blends gain only when they lean heavily toward LDA ($w \geq 0.7$).
- Dropping the 2 lowest-variance standardized components gives a small CV gain over plain LDA (0.251 vs 0.266). This is about half an SE, so the top four candidates are statistically indistinguishable.
- The model beats the majority-class rule by about 8 points on the test set, but it misses half of the actual cases (20 of 40). The 0.5 cutoff ignores the higher cost of a missed diagnosis.
- Limitations: one 116-case test set (wide CI); selecting the minimum of 31 CV scores is optimistic; LDA/QDA assume normal predictors, but `famhist` is binary and `tobacco` and `alcohol` are skewed.

Skill file, invocation, and review

`SKILL.md` (4,690 bytes, under the 5,000-byte limit). Explicit-only invocation uses two safeguards. (1) The front matter sets `disable-model-invocation: true`. This is Claude Code's setting that keeps the agent from loading the skill automatically, so it runs only when called by name (`/lda-qda-report`). (2) The file is stored in `homework/week-07/`, not in any `.claude/skills/` folder, so it is not auto-discovered and is supplied only in an explicit request. Prompt used to apply it:

> *Explicitly apply the skill in `homework/week-07/SKILL.md` (lda-qda-report). Data: `data/saheart.csv`. Response: `chd` (0/1). Predictors: the other nine columns as numeric, with `famhist` coded 1 = Absent, 2 = Present. Seed: 432. Seek the best predictive performance among the LDA/QDA candidates the skill allows, and include plain LDA and QDA as comparisons.*

Review and revision. I checked the agent's output against the training/CV results only. The first version of the skill used a one-SE rule "prefer a simpler candidate," but "simpler" was ambiguous across PCA dimension, LDA vs QDA, and blend weight, and the CV table had several candidates within one SE. I changed the rule to pick the minimum CV error with explicit tie-breaks, and to add a *Within 1 SE* column to the table so the reader can see the near-ties. This revision was decided before the test set was used. I also confirmed that the folds, PCA fitting inside folds, and grid edges matched the skill (the best $q = 7$ is inside 1–9). The test result led to no changes.

```
---
name: lda-qda-report
description: Run a reproducible LDA/QDA classification analysis and write a one-page report. Use ONLY
when I explicitly invoke this skill by name and supply the dataset, response, and predictors; never
load it on your own.
disable-model-invocation: true
---

# LDA/QDA analysis with a one-page report

You are a careful statistician. The user supplies a data file, the response,
the predictors with their coding, and a random seed (pick and state one if
none is given). Do not drop, recode, or add variables beyond what is supplied.
Use Python (scikit-learn) or R (MASS); keep all code in one reproducible script.

## Candidate models (LDA/QDA family only)

1. **LDA** on the supplied predictors (empirical priors).
2. **QDA** on the supplied predictors (empirical priors).
3. **PCA + LDA** and **PCA + QDA**: standardize, project onto the first q
   principal components, then fit. Tune q over 1 to p (or a coarse grid if p
   is large).
4. **Blend**: P(k | x) = w P_LDA(k | x) + (1 - w) P_QDA(k | x), predicting
   the class with the largest blended probability. Tune w over
   {0, 0.1, ..., 1}; optionally blend the best PCA versions.

No other classifier families, no regularization, no tuning of the decision
threshold: predict the class with the largest posterior probability.

## Procedure (no leakage)

1. Set the seed once. Reserve a **stratified 25% test set** before any
   exploration, tuning, or plotting of the response. Do not touch it again
   until step 5.
2. Create **one stratified 10-fold split** of the training set and reuse the
   same folds for every candidate and every tuning value.
```

```
3. Fit all preprocessing (standardization, PCA) **inside each fold** on that
   fold's training part only (e.g. an sklearn `Pipeline`).
4. For each candidate record the mean CV misclassification rate and its
   standard error (SD of the 10 fold errors / sqrt(10)). Within each family
   keep the tuning value with the lowest mean CV error (ties: smaller q, or w
   closer to 0 or 1). Select the candidate with the lowest mean CV error
   overall (ties: the one with fewer parameters, LDA < QDA). Do not apply a
   one-SE rule; instead flag in the table every candidate within one SE of
   the selected one, since those are not clearly worse.
5. Refit the selected model and its preprocessing on the **full training
   set**, then evaluate it **once** on the test set. Report the test
   misclassification rate, a 95% Wilson interval, and the test confusion
   matrix (observed in rows, predicted in columns).
6. Also report the majority-class (no-information) error on the test set and
   the test errors of plain LDA and QDA refit on the full training set as
   reference points, clearly labelled as not used for selection.

Sanity checks before writing: class counts per fold, no missing values,
covariance matrices non-singular (report if QDA warns), selected setting
lies inside its grid (widen the grid if it sits on an edge that can grow).

## Report (at most ONE printed letter page, render and check)

Title, then these short parts, in this order:

1. **Data and goal** (2-3 sentences): n, number of predictors, class
   proportions, split sizes, seed.
2. **Candidates and CV** (2-3 sentences): models, grids, fold scheme,
   selection rule.
3. **Figure** (one, ~6.5 x 2.6 in, 200+ dpi): mean CV error (y, "CV
   misclassification rate") against the tuning parameter (x), one panel per
   tuned family (PCA q; blend weight w), with +/-1 SE bars; draw plain LDA
   and QDA CV errors as labelled horizontal reference lines and mark the
   selected point. Okabe-Ito colors with distinct markers/line styles,
   sentence-case titles, all text >= 9 pt, no top/right spines.
4. **Table** (one): each candidate at its best setting with columns
   *Model, Setting, CV error, SE, Within 1 SE*, sorted by CV error, the
   selected row in bold. Round errors to 3 decimals.
5. **Selected model and test result**: one sentence naming the model and
   setting, then test error (95% CI), the 2x2 (or KxK) test confusion matrix,
   and the reference errors from step 6.
6. **Findings and limitations** (3-5 bullets): what drives performance
   (e.g. shared vs. separate covariance, dimension reduction), how close the
   candidates are relative to their SEs, and limitations (single split, small
   test set, normality, selection noise, threshold 0.5 ignores costs).

Style: plain words, no code in the report, numbers with consistent rounding,
no claims that the numbers do not support. Never revise the model, grid, or
text in response to the test result. After rendering, confirm the report fits
on one page; if not, shorten text first, then shrink the figure.
```

Analysis code. The script that produced the report, `q4_analysis.py`, is submitted next to `SKILL.md` in `homework/week-07/` (plotting and table-formatting code is in this report's source).